UNIVERSITY SCHOOL OF AUTOMATION & ROBOTICS (USAR)

Guru Gobind Singh Indraprastha University, East Delhi Campus,

Surajmal Vihar, Delhi 110092

Embedded Systems Lab File

(ARI 356)

Submitted To:

Dr. Khyati Chopra

Assistant Professor, USAR

Submitted By:

Jatin Gupta

IIOT B2

11519051723

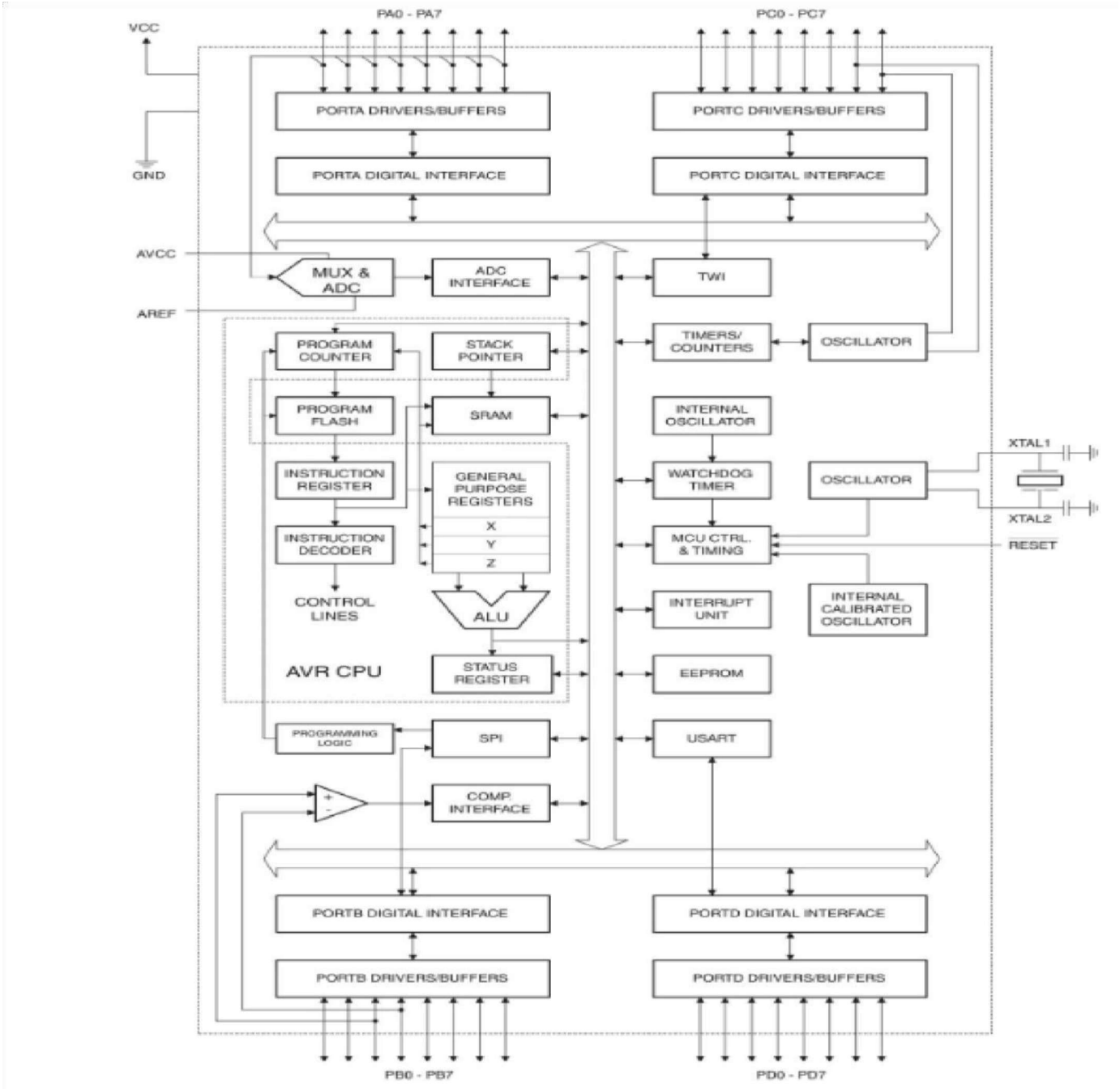
INDEX

S. No.	Experiment	Date	Sign

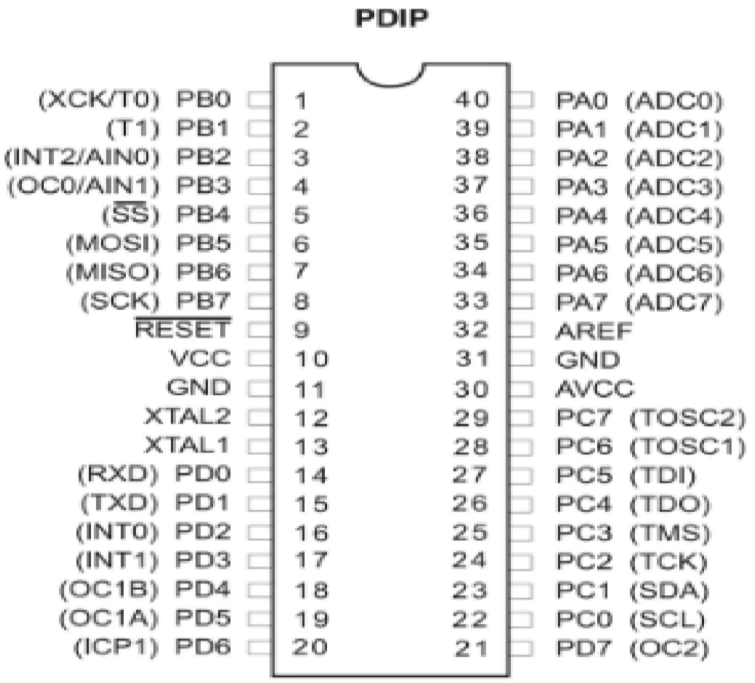
INTRODUCTION TO AVR

1. AVR is a family of microcontrollers developed since 1996 by Atmel, acquired by Microchip Technology in 2016
2. AVR stands for Advanced Virtual RISC.
3. Its architecture was developed by Alf Egil Bogen and Vegard's Wollan.

AVR BLOCK DIAGRAM



AVR PIN OUT DIAGRAM



Features of AVR

1. High-performance, Low-power Atmel AVR 8-bit Microcontroller
2. 32 x 8 General Purpose Working Registers
3. Fully Static Operation
4. Up to 16 MIPS Throughput at 16 MHz
5. 32KB of In-System Self-programmable Flash program memory and 1024B EEPROM
6. 2KB Internal SRAM
7. Operating voltage 4.5V - 5.5V for ATmega32
8. Two 8-bit Timer/Counters with Separate Prescalers and Compare Modes
9. One 16-bit Timer/Counter with Separate Prescaler, Compare Mode, and Capture Mode
10. Real Time Counter with Separate Oscillator
11. Four PWM Channels
12. 8-channel, 10-bit ADC
13. 8 Single-ended Channels
14. 7 Differential Channels in TQFP Package Only
15. 2 Differential Channels with Programmable Gain at 1x, 10x, or 200x Byte-oriented Two-wire
    a. Serial Interface
16. Programmable Serial USART
17. Master/Slave SPI Serial Interface
18. Programmable Watchdog Timer with Separate On-chip Oscillator
19. On-chip Analog Comparator
20. 32 Programmable I/O Lines

EXPERIMENT - 1

AIM: To Write assembly language program for AVR microcontroller to perform the following and analyze the contents of SREG register after each programming:

1. **25H + F2H**
2. **D2H-9FH**
3. **0x77 x 0x34**
4. **0x64/0x04**
5. **AND 0x70, 0x6B**
6. **XOR 0xAA, 0x95**

a. 43H +F2H

CODE:

```
.INCLUDE "M32DEF.INC"
 .ORG 0x00
LDI R20,0x25
LDI R21,0xF2
ADD R20, R21
```

```
; Replace with your application code
.INCLUDE "M32DEF.INC"
.ORG 0X00
LDI R20,0x25
LDI R21,0x25
ADD R20, R21
```

OUTPUT:

Registers

```
R00 = 0x00 R01 = 0x00 R02 = 0x00 R03 = 0x00 R04 = 0x00 R05 = 0x00 R06 = 0x00 R07 = 0x00 R08 = 0x00
  R09 = 0x00 R10 = 0x00 R11 = 0x00 R12 = 0x00 R13 = 0x00 R14 = 0x00 R15 = 0x00 R16 = 0x00 R17 = 0x00
   R18 = 0x00 R19 = 0x00 R20 = 0x17 R21 = 0xF2 R22 = 0x00 R23 = 0x00 R24 = 0x00 R25 = 0x00
  R26 = 0x00 R27 = 0x00 R28 = 0x00 R29 = 0x00 R30 = 0x00 R31 = 0x00
```

Name	Value
Z Register	0x0000
Status Register	I T H S V N Z C
Cycle Counter	2
Frequency	1.000 MHz
Stop Watch	2.00 µs
Registers	
R00	0x00
R01	0x00
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00
R09	0x00
R10	0x00
R11	0x00
R12	0x00
R13	0x00
R14	0x00
R15	0x00
R16	0x00
R17	0x00
R18	0x00
R19	0x00
R20	0x25
R21	0xF2
R22	0x00

Name	Value
Z Register	0x0000
Status Register	I T H S V N Z C
Cycle Counter	647431
Frequency	1.000 MHz
Stop Watch	6,47,431.00 µs
Registers	
R00	0x00
R01	0x00
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00
R09	0x00
R10	0x00
R11	0x00
R12	0x00
R13	0x00
R14	0x00
R15	0x00
R16	0x00
R17	0x00
R18	0x00
R19	0x00
R20	0x17
R21	0xF2
R22	0x00

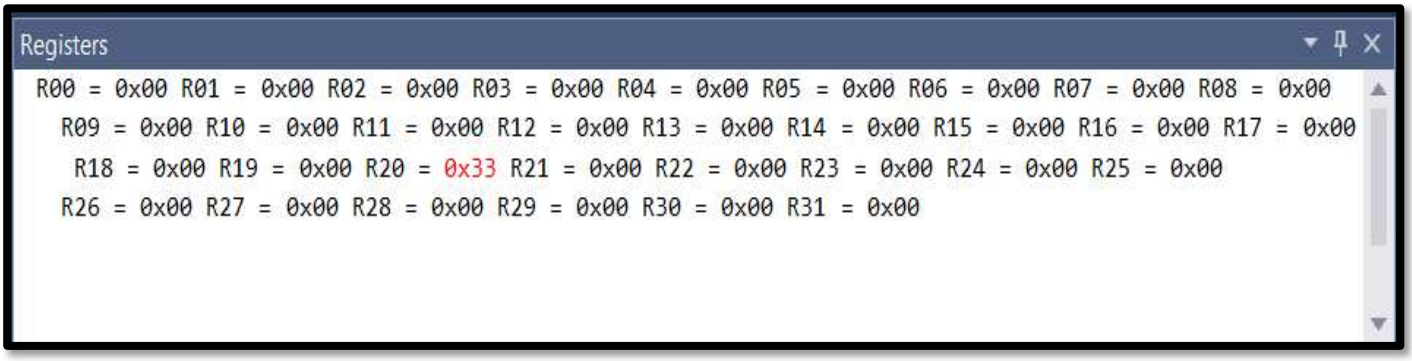
b. D2H-9FH

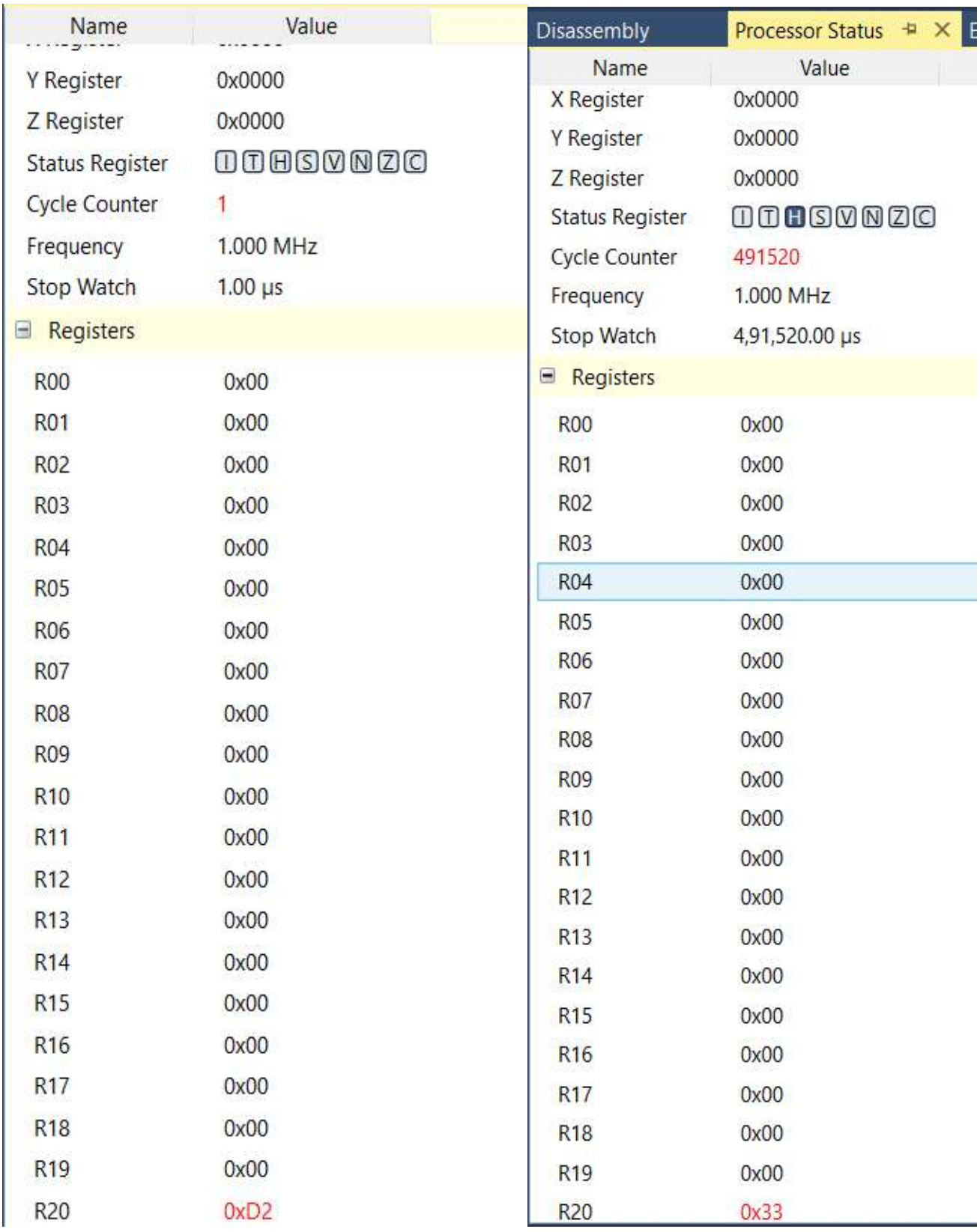
CODE:

```
.INCLUDE "M32DEF.INC"
.ORG 0x00
LDI R20,0xD2
SUBI R20,0x9F
```

```
; Replace with your application code
.INCLUDE "M32DEF.INC"
.ORG 0X00
LDI R20,0xD2
SUBI R20,0x9F
```

OUTPUT:





Name	Value
Y Register	0x0000
Z Register	0x0000
Status Register	I T H S V N Z C
Cycle Counter	1
Frequency	1.000 MHz
Stop Watch	1.00 µs
Registers	
R00	0x00
R01	0x00
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00
R09	0x00
R10	0x00
R11	0x00
R12	0x00
R13	0x00
R14	0x00
R15	0x00
R16	0x00
R17	0x00
R18	0x00
R19	0x00
R20	0xD2

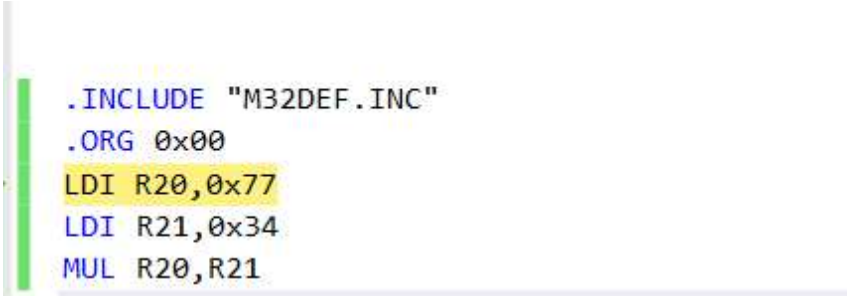
Disassembly | Processor Status

Name	Value
X Register	0x0000
Y Register	0x0000
Z Register	0x0000
Status Register	I T H S V N Z C
Cycle Counter	491520
Frequency	1.000 MHz
Stop Watch	4,91,520.00 µs
Registers	
R00	0x00
R01	0x00
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00
R09	0x00
R10	0x00
R11	0x00
R12	0x00
R13	0x00
R14	0x00
R15	0x00
R16	0x00
R17	0x00
R18	0x00
R19	0x00
R20	0x33

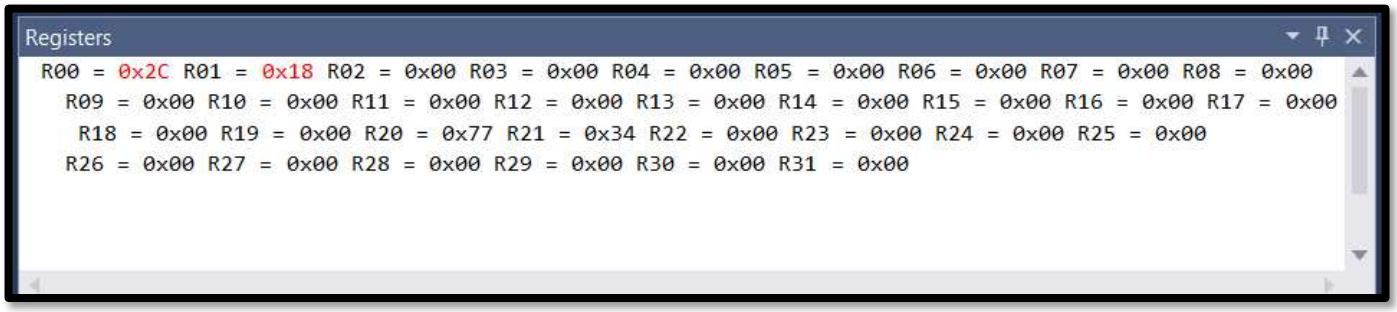
c. 0x77 x 0x34

CODE:

```
.INCLUDE "M32DEF.INC"
.ORG 0x00
LDI R20,0x77
LDI R21,0x34
MUL R20,R21
```



OUTPUT:



Name	Value
Y Register	0x0000
Z Register	0x0000
Status Register	I T H S V N Z C
Cycle Counter	1
Frequency	1.000 MHz
Stop Watch	1.00 µs
Registers	
R00	0x00
R01	0x00
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00
R09	0x00
R10	0x00
R11	0x00
R12	0x00
R13	0x00
R14	0x00
R15	0x00
R16	0x00
R17	0x00
R18	0x00
R19	0x00
R20	0x77

Name	Value
Status Register	I T H S V N Z C
Cycle Counter	313148
Frequency	1.000 MHz
Stop Watch	3,13,148.00 µs
Registers	
R00	0x2C
R01	0x18
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00
R09	0x00
R10	0x00
R11	0x00
R12	0x00
R13	0x00
R14	0x00
R15	0x00
R16	0x00
R17	0x00
R18	0x00
R19	0x00
R20	0x77
R21	0x34
R22	0x00

d. 0x64/0x04

CODE:

```
.INCLUDE "M32DEF.INC"
.ORG 0x0
.DEF NUM=R20
.DEF DENOMINATOR=R21
.DEF QUOTIENT=R22
LDI NUM,0X64
LDI DENOMINATOR,0X04
CLR QUOTIENT
L1: INC QUOTIENT
    SUB NUM,DENOMINATOR
    BRCC L1
    DEC QUOTIENT
    ADD NUM,DENOMINATOR
HERE: JMP HERE
```

```
.INCLUDE "M32DEF.INC"
.ORG 0x0

.DEF NUM=R20
.DEF DENOMINATOR=R21
.DEF QUOTIENT=R22
LDI NUM,0X64
LDI DENOMINATOR,0X04
CLR QUOTIENT
L1: INC QUOTIENT
     SUB NUM,DENOMINATOR
     BRCC L1
     DEC QUOTIENT
     ADD NUM,DENOMINATOR
HERE: JMP HERE
```

OUTPUT:

Registers

```
R00 = 0x00 R01 = 0x00 R02 = 0x00 R03 = 0x00 R04 = 0x00 R05 = 0x00 R06 = 0x00 R07 = 0x00 R08 = 0x00
  R09 = 0x00 R10 = 0x00 R11 = 0x00 R12 = 0x00 R13 = 0x00 R14 = 0x00 R15 = 0x00 R16 = 0x00 R17 = 0x00
   R18 = 0x00 R19 = 0x00 R20 = 0x00 R21 = 0x04 R22 = 0x19 R23 = 0x00 R24 = 0x00 R25 = 0x00
  R26 = 0x00 R27 = 0x00 R28 = 0x00 R29 = 0x00 R30 = 0x00 R31 = 0x00
```

Name	Value
X Register	0x0000
Y Register	0x0000
Z Register	0x0000
Status Register	I T H S V N Z C
Cycle Counter	1
Frequency	1.000 MHz
Stop Watch	1.00 µs
Registers	
R00	0x00
R01	0x00
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00
R09	0x00
R10	0x00
R11	0x00
R12	0x00
R13	0x00
R14	0x00
R15	0x00
R16	0x00
R17	0x00
R18	0x00
R19	0x00
R20	0x64

Name	Value
Z Register	0x0000
Status Register	I T H S V N Z C
Cycle Counter	319896
Frequency	1.000 MHz
Stop Watch	3,19,896.00 µs
Registers	
R00	0x00
R01	0x00
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00
R09	0x00
R10	0x00
R11	0x00
R12	0x00
R13	0x00
R14	0x00
R15	0x00
R16	0x00
R17	0x00
R18	0x00
R19	0x00
R20	0x00
R21	0x04
R22	0x19

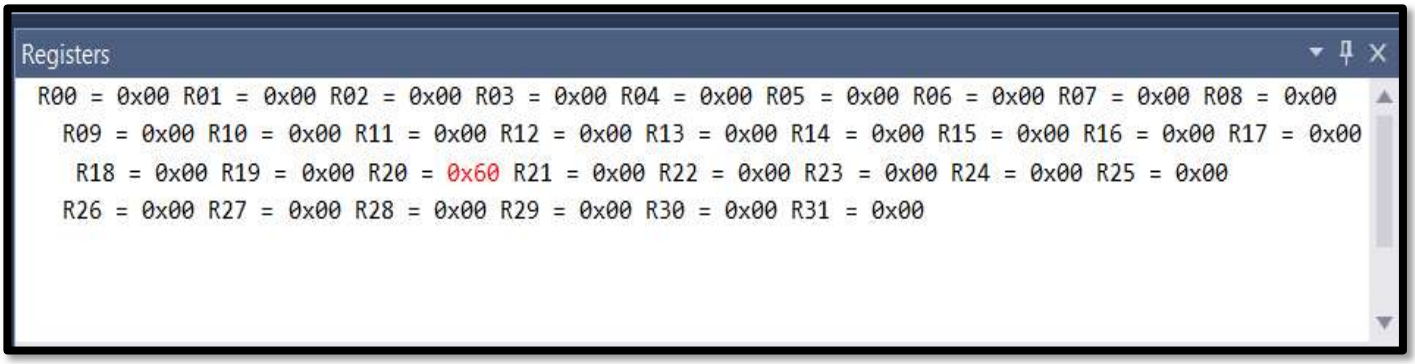
e. AND 0x70, 0x6B

CODE:

```
.INCLUDE "M32DEF.INC"
.ORG 0x0
LDI R20,0X70
ANDI R20,0X6B
```

```
.INCLUDE "M32DEF.INC"
.ORG 0x0
LDI R20,0X70
ANDI R20,0X6B
```

OUTPUT:



Name	Value
X Register	0x0000
Y Register	0x0000
Z Register	0x0000
Status Register	I T H S V N Z C
Cycle Counter	1
Frequency	1.000 MHz
Stop Watch	1.00 µs
Registers	
R00	0x00
R01	0x00
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00
R09	0x00
R10	0x00
R11	0x00
R12	0x00
R13	0x00
R14	0x00
R15	0x00
R16	0x00
R17	0x00
R18	0x00
R19	0x00
R20	0x70

Name	Value
Z Register	0x0000
Status Register	I T H S V N Z C
Cycle Counter	393216
Frequency	1.000 MHz
Stop Watch	3,93,216.00 µs
Registers	
R00	0x00
R01	0x00
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00
R09	0x00
R10	0x00
R11	0x00
R12	0x00
R13	0x00
R14	0x00
R15	0x00
R16	0x00
R17	0x00
R18	0x00
R19	0x00
R20	0x60
R21	0x00
R22	0x00

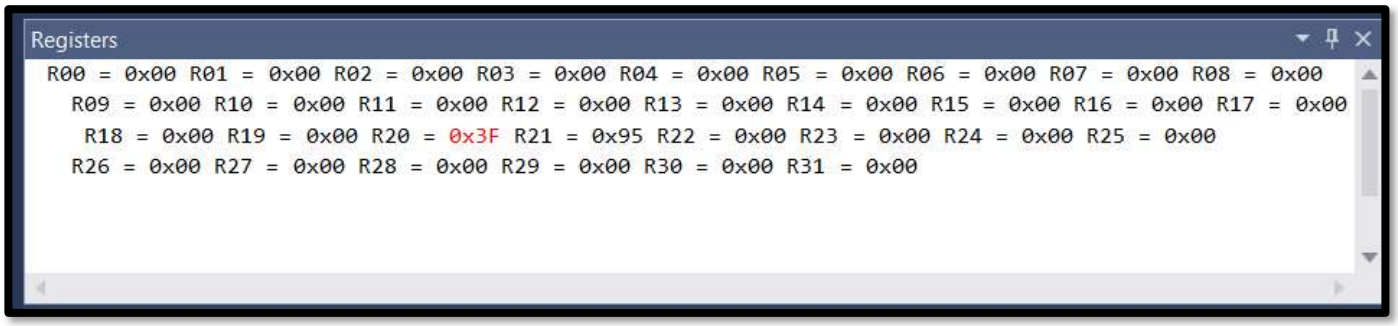
f. XOR 0xAA, 0x95

CODE:

```
.INCLUDE "M32DEF.INC"
.ORG 0x0
 LDI R20,0XAA
LDI R21,0X95
 EOR R20,R21
```

```
.INCLUDE "M32DEF.INC"
.ORG 0x0
LDI R20,0XAA
LDI R21,0X95
EOR R20,R21
```

OUTPUT:



Name	Value
Z Register	0x0000
Status Register	I T H S V N Z C
Cycle Counter	2
Frequency	1.000 MHz
Stop Watch	2.00 µs
Registers	
R00	0x00
R01	0x00
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00
R09	0x00
R10	0x00
R11	0x00
R12	0x00
R13	0x00
R14	0x00
R15	0x00
R16	0x00
R17	0x00
R18	0x00
R19	0x00
R20	0xAA
R21	0x95

Name	Value
X Register	0x0000
Y Register	0x0000
Z Register	0x0000
Status Register	I T H S V N Z C
Cycle Counter	16384
Frequency	1.000 MHz
Stop Watch	16,384.00 µs
Registers	
R00	0x00
R01	0x00
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00
R09	0x00
R10	0x00
R11	0x00
R12	0x00
R13	0x00
R14	0x00
R15	0x00
R16	0x00
R17	0x00
R18	0x00
R19	0x00
R20	0x3F

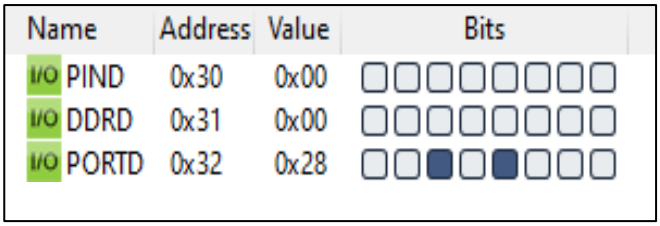
EXPERIMENT 2

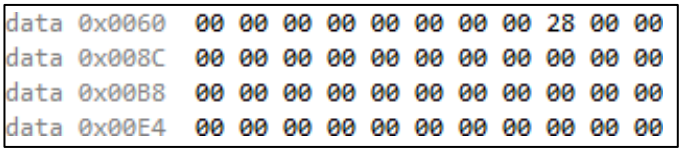
Aim: Write assembly language program for AVR microcontroller to copy data from location $68 to Port D using R19.

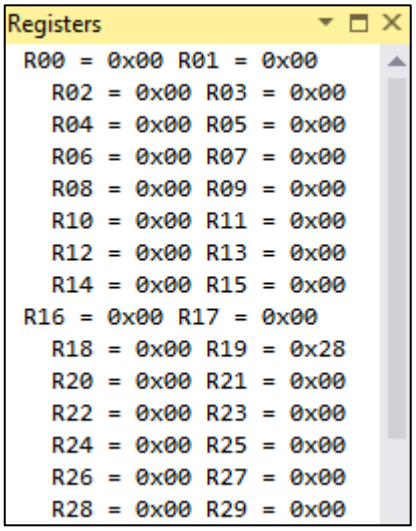
Code:

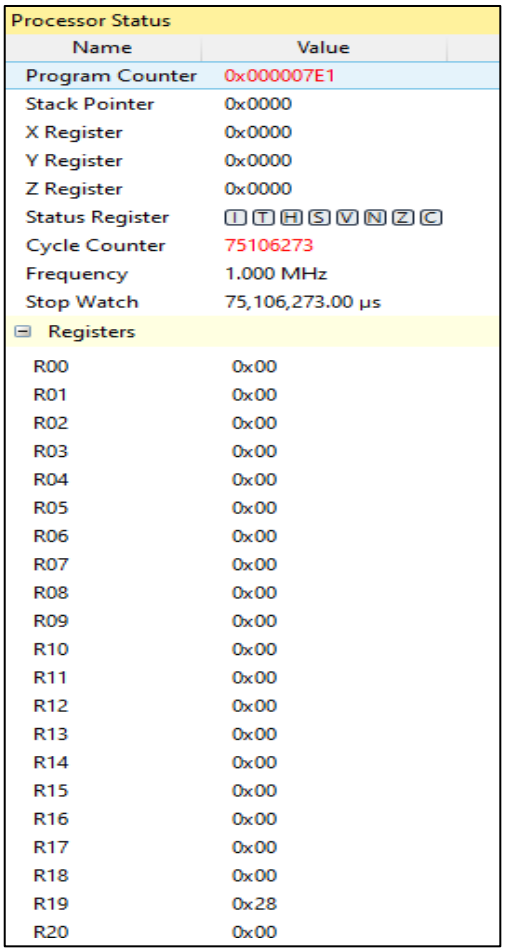
```
.INCLUDE "M32DEF.INC"
.ORG 0x0
LDI R19, $28
STS $68, R19
LDS R19,$68
OUT PORTD, R19
```

Output:






Registers
R00 = 0x00 R01 = 0x00
R02 = 0x00 R03 = 0x00
R04 = 0x00 R05 = 0x00
R06 = 0x00 R07 = 0x00
R08 = 0x00 R09 = 0x00
R10 = 0x00 R11 = 0x00
R12 = 0x00 R13 = 0x00
R14 = 0x00 R15 = 0x00
R16 = 0x00 R17 = 0x00
R18 = 0x00 R19 = 0x28
R20 = 0x00 R21 = 0x00
R22 = 0x00 R23 = 0x00
R24 = 0x00 R25 = 0x00
R26 = 0x00 R27 = 0x00
R28 = 0x00 R29 = 0x00


Processor Status
Name
Value
Program Counter
0x000007E1
Stack Pointer
0x0000
X Register
0x0000
Y Register
0x0000
Z Register
0x0000
Status Register
I T H S V N Z C
Cycle Counter
75106273
Frequency
1.000 MHz
Stop Watch
75,106,273.00 µs
Registers
R00 0x00
R01 0x00
R02 0x00
R03 0x00
R04 0x00
R05 0x00
R06 0x00
R07 0x00
R08 0x00
R09 0x00
R10 0x00
R11 0x00
R12 0x00
R13 0x00
R14 0x00
R15 0x00
R16 0x00
R17 0x00
R18 0x00
R19 0x28
R20 0x00

EXPERIMENT 3

Aim: Write assembly language program for AVR microcontroller to toggle all bits of PORTB by sending it to $55 and $AAH continuously. Put a time delay between each issuing of data to port B

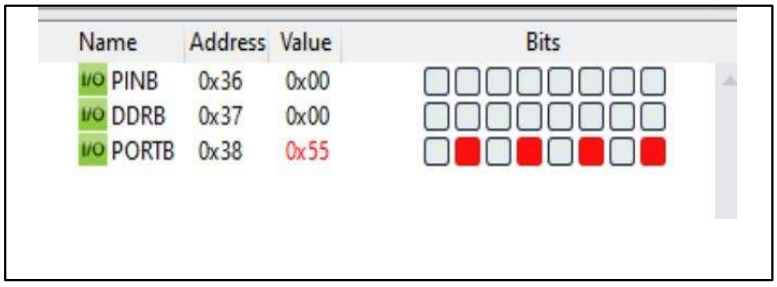
Code:

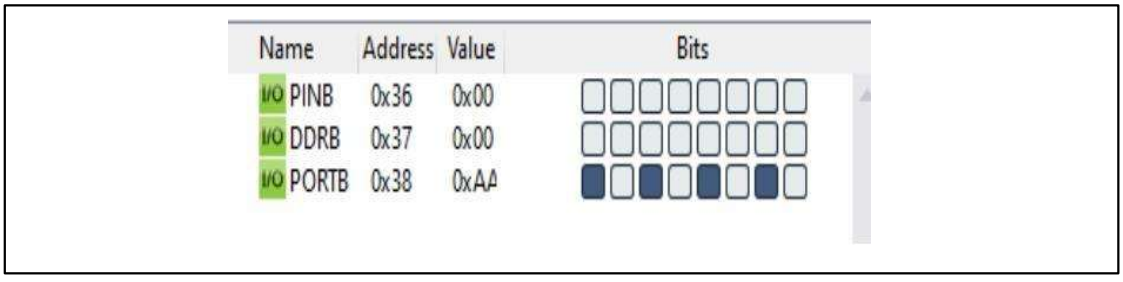
```
INCLUDE "M32DEF.INC"
.ORG 0x0

LDI R16,HIGH(RAMEND)
OUT SPH,R16
LDI R16,LOW(RAMEND)
OUT SPL,R16

BACK:
        LDI R16,0X55
        OUT PORTB,R16
        CALL DELAY
        LDI R16,0XAA
        OUT PORTB,R16
        CALL DELAY
        RJMP BACK

        .ORG 0X300
DELAY:
         LDI R20,0XFF
AGAIN:
         NOP
         NOP
         DEC R20
         BRNE AGAIN
         RET
```

main.asm*

```
INCLUDE "M32DEF.INC"
.ORG 0x0

LDI R16,HIGH(RAMEND)
OUT SPH,R16
LDI R16,LOW(RAMEND)
OUT SPL,R16

BACK:
        LDI R16,0X55
        OUT PORTB,R16
        CALL DELAY
        LDI R16,0XAA
        OUT PORTB,R16
        CALL DELAY
        RJMP BACK

        .ORG 0X300
DELAY:
         LDI R20,0XFF
AGAIN:
         NOP
         NOP
         DEC R20
         BRNE AGAIN
         RET
```

Output:





EXPERIMENT 4

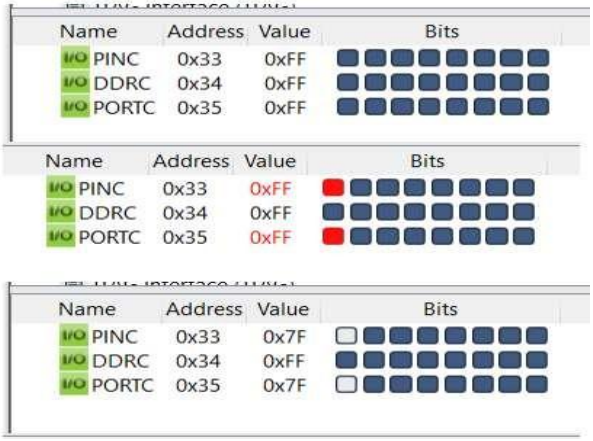
Aim: An LED is connected to each pin of port C. Write an assembly program to turn on each pin from pin C0 to C7. Call a delay subroutine before turning on next LED.

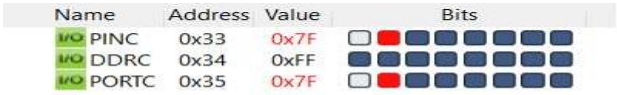
Code:

```
INCLUDE "M32DEF.INC"
.ORG 0x0
LDI R16,HIGH(RAMEND)
OUT SPH,R16
LDI R16,LOW(RAMEND)
OUT SPL,R16
LDI R16,0XFF
OUT DDRC,R16
SBI PORTC,0
CALL DELAY
SBI PORTC,1
CALL DELAY
SBI PORTC,2
CALL DELAY
SBI PORTC,3
CALL DELAY
SBI PORTC,4
CALL DELAY
SBI PORTC,5
CALL DELAY
SBI PORTC,6
CALL DELAY
SBI PORTC,7
CALL DELAY
.ORG 0X300

DELAY:
        LDI R20,0X05

AGAIN:
        NOP
        NOP
        DEC R20
        BRNE AGAIN RET
```

```
; Replace with your application code
INCLUDE "M32DEF.INC"
.ORG 0x0
LDI R16,HIGH(RAMEND)
OUT SPH,R16
LDI R16,LOW(RAMEND)
OUT SPL,R16
LDI R16,0XFF
OUT DDRC,R16
SBI PORTC,0
CALL DELAY
SBI PORTC,1
CALL DELAY
SBI PORTC,2
CALL DELAY
SBI PORTC,3
CALL DELAY
SBI PORTC,4
CALL DELAY
SBI PORTC,5
CALL DELAY
SBI PORTC,6
CALL DELAY
SBI PORTC,7
CALL DELAY
.ORG 0X300

DELAY:
LDI R20,0X05

AGAIN:
NOP
NOP
DEC R20
BRNE AGAIN RET
```

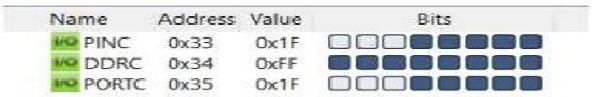
Output:



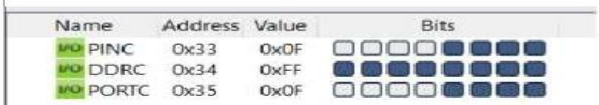

Name Address Value Bits
PINC 0x33 0x7F
DDRC 0x34 0xFF
PORTC 0x35 0x7F

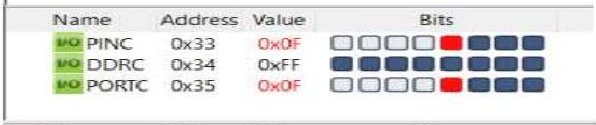

Name Address Value Bits
PINC 0x33 0x3F
DDRC 0x34 0xFF
PORTC 0x35 0x3F


Name Address Value Bits
PINC 0x33 0x3F
DDRC 0x34 0xFF
PORTC 0x35 0x3F


Name Address Value Bits
PINC 0x33 0x1F
DDRC 0x34 0xFF
PORTC 0x35 0x1F

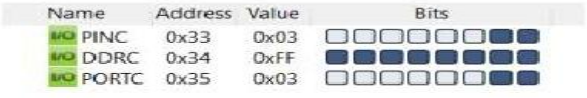

Name Address Value Bits
PINC 0x33 0x1F
DDRC 0x34 0xFF
PORTC 0x35 0x1F

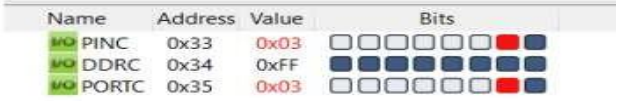

Name Address Value Bits
PINC 0x33 0x0F
DDRC 0x34 0xFF
PORTC 0x35 0x0F

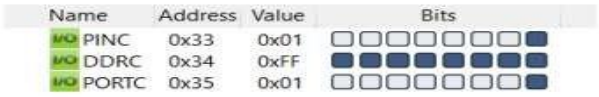

Name Address Value Bits
PINC 0x33 0x0F
DDRC 0x34 0xFF
PORTC 0x35 0x0F

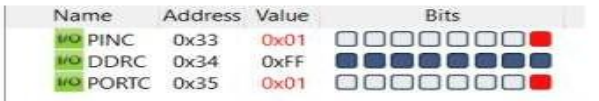

Name Address Value Bits
PINC 0x33 0x07
DDRC 0x34 0xFF
PORTC 0x35 0x07

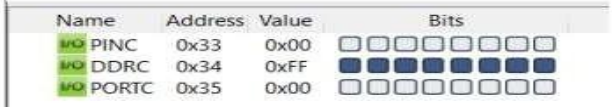

Name Address Value Bits
PINC 0x33 0x07
DDRC 0x34 0xFF
PORTC 0x35 0x07


Name Address Value Bits
PINC 0x33 0x03
DDRC 0x34 0xFF
PORTC 0x35 0x03


Name Address Value Bits
PINC 0x33 0x03
DDRC 0x34 0xFF
PORTC 0x35 0x03


Name Address Value Bits
PINC 0x33 0x01
DDRC 0x34 0xFF
PORTC 0x35 0x01


Name Address Value Bits
PINC 0x33 0x01
DDRC 0x34 0xFF
PORTC 0x35 0x01


Name Address Value Bits
PINC 0x33 0x00
DDRC 0x34 0xFF
PORTC 0x35 0x00


Name Address Value Bits
PINC 0x33 0x00
DDRC 0x34 0xFF
PORTC 0x35 0x00

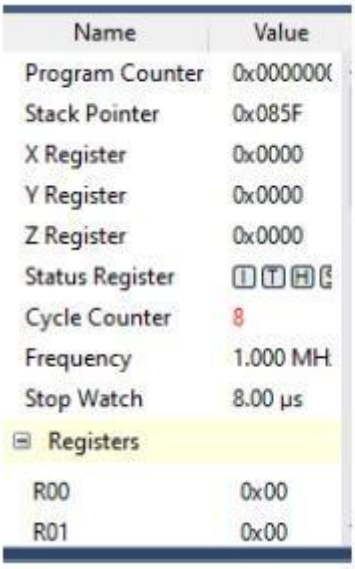
Experiment No: 5

Aim: Write an assembly program to create a square wave of 66% duty cycle on bit 2 of port C.

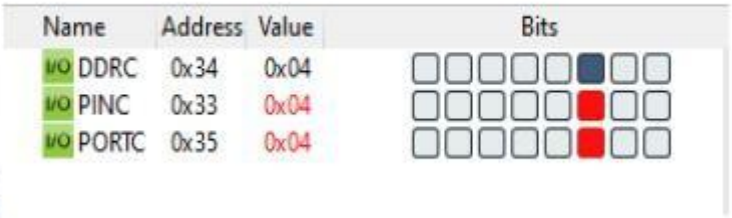
```
INCLUDE "M32DEF.INC"
.ORG 0x0

LDI R16,HIGH(RAMEND)
OUT SPH,R16
LDI R16,LOW(RAMEND)
OUT SPL,R16
SBI DDRC,2

HERE: SBI PORTC,2
CALL DELAY
CALL DELAY
CBI PORTC,2
CALL DELAY
RJMP HERE

.ORG 0X300
DELAY: LDI R20,0X05
AGAIN:
NOP
NOP
DEC R20
BRNE AGAIN RET
```

```
; Replace with your application code
INCLUDE "M32DEF.INC"
.ORG 0x0

LDI R16,HIGH(RAMEND)
OUT SPH,R16
LDI R16,LOW(RAMEND)
OUT SPL,R16
SBI DDRC,2

HERE: SBI PORTC,2
CALL DELAY
CALL DELAY
CBI PORTC,2
CALL DELAY
RJMP HERE

.ORG 0X300
DELAY: LDI R20,0X05
AGAIN:
NOP
NOP
DEC R20
BRNE AGAIN RET
```

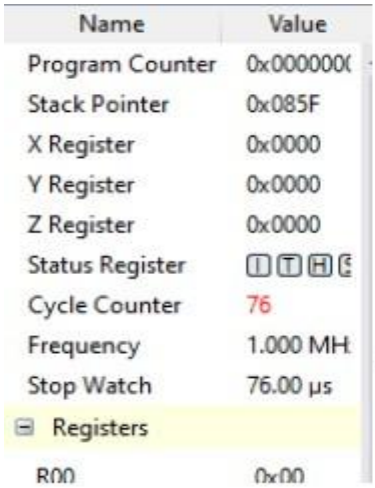
Output:



Name	Value
Program Counter	0x000000(
Stack Pointer	0x085F
X Register	0x0000
Y Register	0x0000
Z Register	0x0000
Status Register	ITHS
Cycle Counter	8
Frequency	1.000 MH
Stop Watch	8.00 µs
Registers	
R00	0x00
R01	0x00

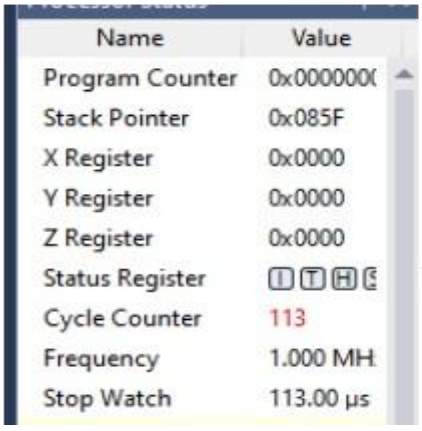


Name	Address	Value	Bits
DDRC	0x34	0x04	
PINC	0x33	0x04	
PORTC	0x35	0x04	



Name	Value
Program Counter	0x000000(
Stack Pointer	0x085F
X Register	0x0000
Y Register	0x0000
Z Register	0x0000
Status Register	ITHS
Cycle Counter	76
Frequency	1.000 MH
Stop Watch	76.00 µs
Registers	
R00	0x00

Name	Address	Value	Bits
DDRC	0x34	0x04	
PINC	0x33	0x00	
PORTC	0x35	0x00	



Name	Value
Program Counter	0x000000(
Stack Pointer	0x085F
X Register	0x0000
Y Register	0x0000
Z Register	0x0000
Status Register	ITHS
Cycle Counter	113
Frequency	1.000 MH
Stop Watch	113.00 µs

Name	Address	Value	Bits
DDRC	0x34	0x04	
PINC	0x33	0x04	
PORTC	0x35	0x04	

Experiment No: 6

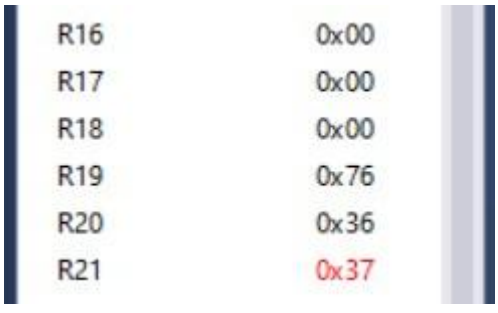
Aim: Write an assembly program to convert following packed BCD numbers to ASCII.
a) 0X76
b) 0X85, Place the ASCII codes into R20 and R21.

c) Code:

LDI R20,0x76 ;the packed BCD to be converted is 76
MOV R19,R20 ;R20 = R19 = 76H
ANDI R20,0x0F ;mask the upper nibble (R20 = 06H)
ORI R20,0x30 ;make it ASCII (R20 = 36H)
MOV R21,R19 ;R21 = R19 = 76H
SWAP R21 ;swap nibbles (R21 = 67H)
ANDI R21,0x0F ;mask the upper nibble (R21 = 07)
ORI R21,0x30 ;make it ASCII (R21 = 37H)

```
; Replace with your application code
INCLUDE "M32DEF.INC"
.ORG 0x0

LDI R20,0x76 ;the packed BCD to be converted is 76
MOV R19,R20 ;R20 = R19 = 76H
ANDI R20,0x0F ;mask the upper nibble (R20 = 06H)
ORI R20,0x30 ;make it ASCII (R20 = 36H)
MOV R21,R19 ;R21 = R19 = 76H
SWAP R21 ;swap nibbles (R21 = 67H)
ANDI R21,0x0F ;mask the upper nibble (R21 = 07)
ORI R21,0x30 ;make it ASCII (R21 = 37H)
```

Output:



b) Code:

LDI R20,0x85 ;the packed BCD to be converted is 85
MOV R19,R20 ;R20 = R19 = 85H
ANDI R20,0x0F ;mask the upper nibble (R20 = 05H)
ORI R20,0x30 ;make it ASCII (R20 = 35H)
MOV R21,R19 ;R21 = R19 = 85H
SWAP R21 ;swap nibbles (R21 = 58H)
ANDI R21,0x0F ;mask the upper nibble (R21 = 08)
ORI R21,0x30 ;make it ASCII (R21 = 38H)

```
; Replace with your application code
INCLUDE "M32DEF.INC"
.ORG 0x0

LDI R20,0x85 ;the packed BCD to be converted is 85
MOV R19,R20 ;R20 = R19 = 85H
ANDI R20,0x0F ;mask the upper nibble (R20 = 05H)
ORI R20,0x30 ;make it ASCII (R20 = 35H)
MOV R21,R19 ;R21 = R19 = 85H
SWAP R21 ;swap nibbles (R21 = 58H)
ANDI R21,0x0F ;mask the upper nibble (R21 = 08)
ORI R21,0x30 ;make it ASCII (R21 = 38H)
```

Output:

R08	0x00
R09	0x00
R10	0x00
R11	0x00
R12	0x00
R13	0x00
R14	0x00
R15	0x00
R16	0x00
R17	0x00
R18	0x00
R19	0x85
R20	0x35
R21	0x38
R22	0x00
R23	0x00
R24	0x00

Experiment No: 7

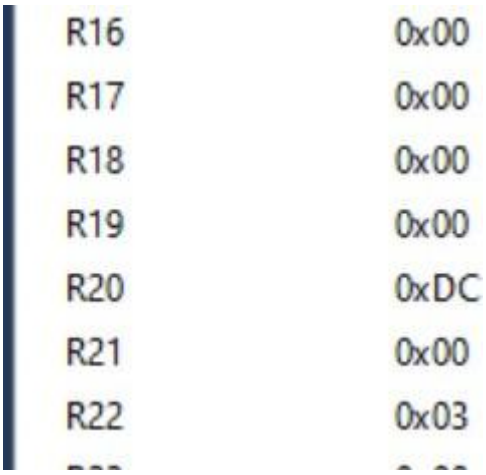
Aim: Write an assembly program that finds number of zero's in a 8-bit data item.

Code:

```
.INCLUDE "M32DEF.INC"
.ORG 0x00
LDI R20,0b10111001
LDI R21, 8 ;set a counter to 8
LDI R22, 0 ;set number of zeros to 0
AGAIN:ROL R20 ;move MSB to Carry
BRCS NEXT ;branch if set carry
INC R22 ;increment number of zeros
NEXT: DEC R21 ;decrement the counter
BRNE AGAIN
```

```
; Replace with your application code
.INCLUDE "M32DEF.INC"
.ORG 0x00
LDI R20,0b10111001
LDI R21, 8 ;set a counter to 8
LDI R22, 0 ;set number of zeros to 0
AGAIN:ROL R20 ;move MSB to Carry
BRCS NEXT ;branch if set carry
INC R22 ;increment number of zeros
NEXT: DEC R21 ;decrement the counter
BRNE AGAIN
```

Output:

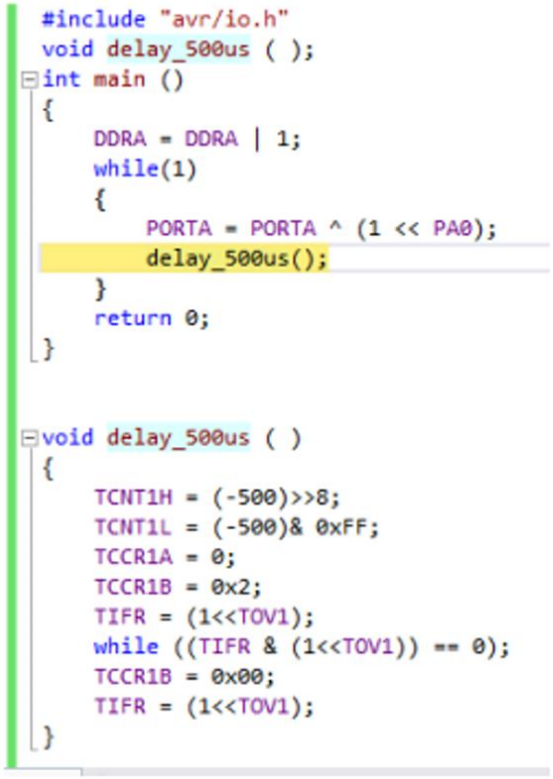


Experiment No: 8

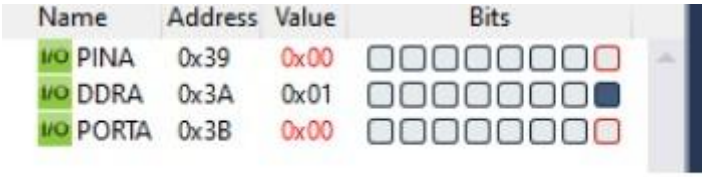
Aim: Assuming XTAL=8MHz, write a C program to generate 1KHz frequency on PC4 using Timers.

Code:

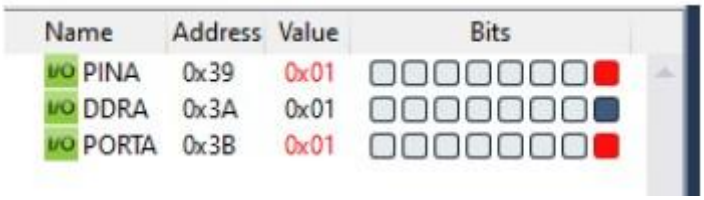
```
#include "avr/io.h"
void delay_500us ( );
int main ()
{
	DDRA = DDRA | 1;
	while(1)
	{
		PORTA = PORTA ^ (1 << PA0);
		delay_500us();
	}
	return 0;
}
void delay_500us ( )
{
	TCNT1H = (-500)>>8;
	TCNT1L = (-500)& 0xFF;
	TCCR1A = 0;
	TCCR1B = 0x2;
	TIFR = (1<<TOV1);
	while ((TIFR & (1<<TOV1)) == 0);
	TCCR1B = 0x00;
	TIFR = (1<<TOV1);
}
```



Output:



Name	Address	Value	Bits
I/O PINA	0x39	0x00	
I/O DDRA	0x3A	0x01	
I/O PORTA	0x3B	0x00	



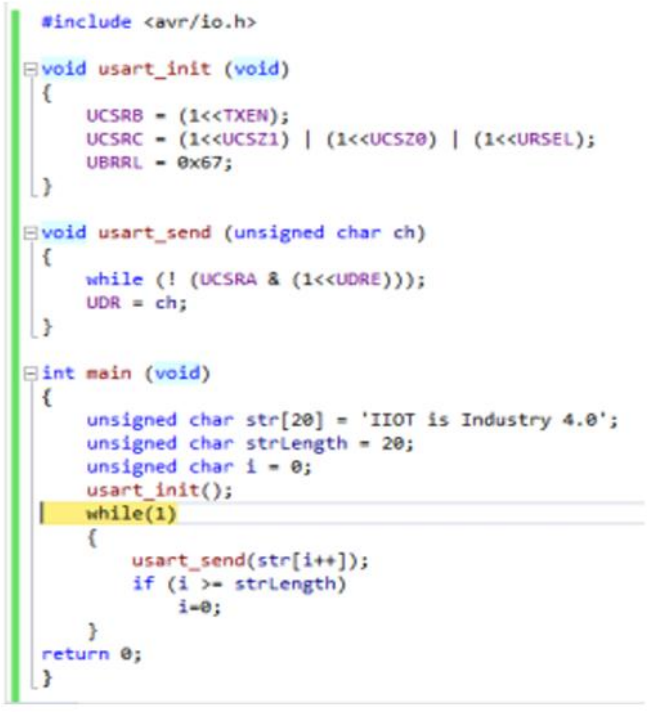
Name	Address	Value	Bits
I/O PINA	0x39	0x01	
I/O DDRA	0x3A	0x01	
I/O PORTA	0x3B	0x01	

Experiment No: 9

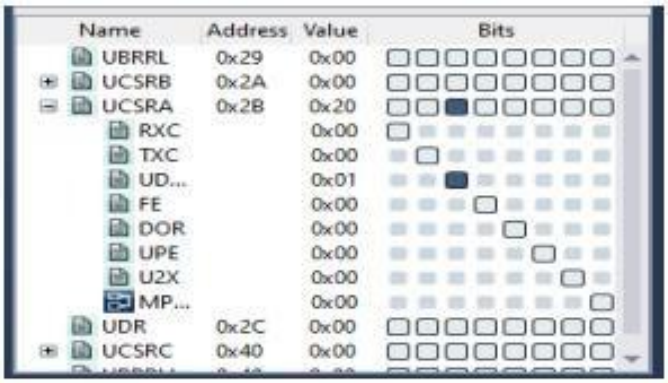
Aim: Write a C program to send the message "IIOT is Industry 4.0" to the serial port continuously at 9600 baud rate. Use 8-bit data and 1 stop bit, assume XTAL=16MHz.

Code:

```
#include <avr/io.h>
void usart_init (void)
{
        UCSRB = (1<<TXEN);
        UCSRC = (1<<UCSZ1) | (1<<UCSZ0) | (1<<URSEL);
        UBRRL = 0x67;
}
void usart_send (unsigned char ch)
{
        while (! (UCSRA & (1<<UDRE)));
        UDR = ch;
}
int main (void)
{
        unsigned char str[20] = "IIOT is Industry 4.0";
        unsigned char strLength = 20;
        unsigned char i = 0;
        usart_init();
        while(1)
        {
                usart_send(str[i++]);
                if (i >= strLength)
                i=0;
        }
        return 0;
}
```



Output:


Name
Address
Value
Bits
UBRRL
0x29
0x00
UCSRB
0x2A
0x00
UCSRA
0x2B
0x20
RXC
0x00
TXC
0x00
UD...
0x01
FE
0x00
DOR
0x00
UPE
0x00
U2X
0x00
MP...
0x00
UDR
0x2C
0x00
UCSRC
0x40
0x00